FIRE HANDBOOK

Find the root cause fast: pick the angle, then pick the tool. Do not re-read code you can run. Measure it.

1 **REPRODUCE** make it fail on demand, as small as you can	**2** **LOCALIZE** which commit, layer, data, or moment? Halve it.	**3** **OBSERVE** debugger, profiler, sanitizer. Facts, not guesses.	**4** **HYPOTHESIZE** one sentence that an experiment could disprove	**5** **TEST** cheapest experiment. Change one thing.
6 **FIX AT ROOT** the decision, not the crash line	**7** **PROVE** failing test goes green; revert fix and it goes red	**8** **RECORD** one paragraph in the journal		

FIVE RULES OF THE ROAD

1. **Run it first.** Build, run, watch what it really does.
2. **Follow the data, not the code.** A watchpoint beats 100 grep hits (S7).
3. **Crash site is not bug site.** Walk backwards from the bad value.
4. **No symbols, no debugger.** `-g` first (S5).
5. **25 minutes, no falsified hypothesis?** Change angle or ask, and bring what you tried (S2).

> **BEFORE YOU INSTALL ANYTHING AT WORK:** ask which tools are already approved. Never paste program code or logs into an AI tool that security has not approved (S14).

60-SECOND TRIAGE

✓ RUN ON THIS MAC

```
# 1 Can I make it fail on demand?   2 Copy the exact error
text.
# 3 Did it ever work?  yes -> git bisect (S4)   4 Symbols?
(S5)
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug \
      -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
cmake --build build -j
./build/app            # run it, see the failure
```

HOW TO READ THE TAGS

✓ **run on this Mac** (zsh, Apple clang, LLDB) while building this handbook. **from docs** came from official docs or the talk and was not run here (no GDB, perf, or valgrind on Apple Silicon). Try those once somewhere safe before you rely on them under pressure. **S#** = sheet number = chapter number in `handbook/chapters/`.

Source: *Understanding Large and Unfamiliar Codebases*, Mike Shah and Chris Croft-White, C++ on Sea 2025, extended with standard best practice.

SHEET INDEX (sheet number = chapter number)

1	**Symptom to angle to tool**	Something is wrong; which kind of problem is it?
2	**The debug loop**	Thrashing; need a process. 25-minute rule, ask-for-help template.
3	**First 30 minutes**	New repo or component: cloc, build, run, find main, find tests.
4	**Search, navigate, history**	Where is it? Who touches it? When did it change? git bisect.
5	**Build and debug symbols**	No symbols, CMake maze, "works in release."
6	**Debugger fundamentals**	GDB and LLDB side by side; break, step, attach, inspect.
7	**Follow the data**	**One variable, 100 places, a message queue.** Watchpoints.
8	**Time-travel debugging**	"How did I get here?" Who wrote it last? Races you can replay.
9	**Profilers and call graphs**	What runs when I do X? Where is the time going?
10	**Structure, docs, dependencies**	Doxygen, include and target graphs, libraries you do not own.
11	**Shrink the problem**	Tests, thin slices, first tasks, minimal repros.
12	**Crashes, memory, hangs, races**	Sanitizers, core dumps, thread dumps, valgrind.
13	**Workspace and journal**	tmux, vim, the journal template, shadowing a senior.
14	**AI: trust, but verify**	Turn an AI claim into an experiment. Check what is allowed first.
15	**Links and next steps**	Primary sources, books, a practice plan.

1 SYMPTOM TO ANGLE TO TOOL

USE WHEN something is wrong and you do not yet know what kind of problem it is. Find the row. Do the first move.

YOU SEE	TYPE	FIRST MOVE	TOOL	SHEET
No idea what this repo or program does	Orientation	Build, run it, count code, find `main`	`cloc` `rg` debugger `start`	3
Where is the code for this feature, message, or button?	Locate	Search the *visible string* (error text, label, flag), then walk to callers	`rg` , clangd references	4
What runs when I do X?	Control flow	Break on a function you know is involved; read `bt`	gdb/lldb, `perf record -g`	6 9
Who changes this variable? Where did this value come from?	Data flow	Hardware watchpoint, with a condition on the bad value	`watch -l ... if ...`	7 8
The variable appears 100 times: methods, pointers, messages	Data flow at scale	Do not read it. Watch it. Break at the *producer* (post/send), not the consumer	conditional watch + conditional break	7
Segfault, abort, assert, uncaught exception	Crash	Backtrace first; then work backwards from the bad value	debugger, core, ASan	12 6
Wrong output, no crash	Logic / data	Check the value at each stage boundary; bisect the pipeline	`dprintf` , conditional break	7 8
It worked last week	Regression	Script a pass/fail check, let git find the commit	`git bisect run` , `git log -S`	4
Slow, or CPU pegged	Performance	Sample first, never guess	`perf record -g` , callgrind, `sample`	9
Hangs or freezes	Deadlock / blocked	Attach and dump *every* thread's stack	`thread apply all bt` , `strace -p`	12
Garbage values, corruption, leak, use-after-free	Memory	Rebuild with sanitizers and rerun	`-fsanitize=address` , valgrind	12
Only sometimes, only under load, only in release	Race / heisenbug	Make it deterministic: record once, replay forever	rr, UDB, TSan	8 12
Will not build, link, or CMake is a maze	Build	See the real compiler command; list cache variables	`--verbose` , `cmake -LAH`	5
Undefined reference, symbol not found, wrong library loaded	Link / deps	What does the binary need, and what does it have?	`nm | c++filt` , `ldd` / `otool -L`	10
Third-party or closed library misbehaves	Code I do not own	Watch the boundary: what goes in, what comes out	record API calls, `ltrace` / `strace`	10 8
Cannot hold this class hierarchy in my head	Structure	Generate a map instead of reading	doxygen, clangd call hierarchy	10
Is there already a test for this component?	Shrink scale	Look in `tests/` before touching code; run only that test	`ctest -N` , `ctest -R`	11
The task is huge and I am drowning	Scale	Find the smallest possible change and a tiny example	tests, examples, TODO hunt	11
AI told me the cause and sounds sure	Unverified claim	Make it prove it with debugger evidence	watchpoint / replay as the judge	14
Stuck for 30+ minutes	Process failure	Stop. Write hypotheses. Change angle, or ask with what you tried	journal	2 13

CAN'T TELL WHICH ROW? DECISION TREE

FROM DOCS · TRY ONCE FIRST

```
Fails every time?
 NO  -> make it deterministic first: loop it, fix seeds,
        pin threads, record it                    (S8, S12)
 YES -> Crashes?          bt, then walk BACKWARDS from the
                          bad value                (S12, S7)
        Wrong value?      watch it; who wrote it last? (S7)
        Used to work?     git bisect                   (S4)
        Slow / spinning?  profile                      (S9)
        Stuck / frozen?   attach + all-thread stacks   (S12)
        Don't know where the code lives?
                          run it, grep the visible string,
                          debugger bt                  (S3,
S4)
```

TWELVE COMMANDS TO KNOW COLD

MIXED · MARKED INLINE

```
rg -n -w SYMBOL src/                          # ✓ every use of
a name
rg -n 'TODO|FIXME|HACK' src/                  # ✓ breadcrumbs
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug  # ✓ get -g
gdb --args ./build/app ARGS                   # docs
(gdb) start | bt | frame N | info locals | p *ptr
(gdb) watch -l obj->f if obj->f < 0       # who made it bad
(gdb) break Bus::post if m.value < 0      # catch the sender
(gdb) thread apply all bt                  # hang
git bisect start BAD GOOD && git bisect run ./check.sh # ✓
git log -S'symbol' --oneline              # ✓ when it
appeared
perf record -g -p PID -- sleep 20         # docs
valgrind --leak-check=full --track-origins=yes ./app
```

2 THE DEBUG LOOP

USE WHEN you are thrashing: re-reading the same functions, changing three things between runs. You need a process, not another tool.

STRATEGY

1. **Reproduce.** Fixed input, fixed seed, one thread if you can. If you cannot reproduce it, *making it reproduce* is the task.
2. **Localize by halving.** Pick the axis that halves the search fastest (table below).
3. **Observe with a tool.** Debugger state, a profile, a sanitizer report.
4. **Hypothesize in one sentence** that an experiment can falsify. Good: "`m_total` goes negative in `applyDelta` because `rebalance` posts a delta from a stale `believedTotal_`." Bad: "the messaging is broken."
5. **Test cheaply.** A watchpoint, a conditional break, one `dprintf`. Change one thing at a time.
6. **Fix at the root.** Ask "why?" until the answer is a decision someone made.
7. **Prove it.** Add a test that fails before and passes after. Revert the fix once and confirm the bug returns.
8. **Record** it in the journal (S13).

ANTI-PATTERNS

- Re-reading code instead of running it
- A null check at the crash site
- Three changes between runs
- Trusting a confident explanation, human or AI
- Debugging Debug when the bug is in Release (S5)
- Writing nothing down

WHICH AXIS HALVES THE SEARCH?

AXIS	QUESTION	TOOL
History	Which commit broke it?	`git bisect run`
Code	Which module or class?	break + `bt`, `perf`, grep visible text
Data	Which stage makes the value bad?	conditional break, `dprintf`
Time	When in the run does it first go wrong?	watchpoint, reverse debugging
Config	Debug vs release? Flag? Library version?	rebuild with one change

STUCK 25 TO 30 MINUTES?

The number is a rule of thumb. If no hypothesis has been falsified, you are not progressing: switch angle (data, flow, history), shrink the repro, or ask. Asking is expected. *Bring something to the table.*

ASK-FOR-HELP TEMPLATE

```
Goal:      one line
Repro:     exact command, input, commit
Expected vs actual:
Tried:     1) ... (learned ...)  2) ...
Hypothesis now: one falsifiable sentence
Question:  one concrete thing
Evidence:  bt, log excerpt, journal link
```

Best junior tip from the talk: ask a senior, "Can I shadow you for 15 minutes?" You pick up their tools, not just answers.

3 FIRST 30 MINUTES IN A NEW CODEBASE

USE WHEN day one on a repo, a new component, or "where do I even start?" Goal: a one-page mental map and a working debug build before you read a single function.

STRATEGY (TALK ORDER)

1. **Measure** size and languages.
2. **Layout**: README, `docs/`, `tests/`, top `CMakeLists.txt`.
3. **Build with `-g`.**
4. **Run it** like a user. "Sounds silly, but a high-level understanding of what your program actually does is key."
5. **Anchor on `main()`**. It hints at the architecture: plugins? event loop? Log what happens *before* main (init) and *after* (cleanup).
6. **Find tests** for *your* component.
7. **Read the history** of your area.
8. **Journal entry #1**: size, entry points, build recipe, three open questions.

> **MANY `MAIN`S IS NORMAL.** OpenTTD had about 33 to 37 hits: shaders, third-party test framework, one per OS, plus the real `openttd_main`. A tiny `main` that hands off to a "real main" is a good sign.

MIXED · MARKED INLINE

```
# size and languages
cloc .                      # brew/apt install cloc
cloc --vcs=git              # tracked files only
# layout
tree -L 2 -I 'build|.git|third_party'   # ✓
find . -name CMakeLists.txt | head
# entry points: the talk's exact grep  ✓
grep -irn -I --include='*.cpp' "main()" .
rg -n --type cpp '\bmain\s*\(' .
# ...or let the debugger land there
gdb ./build/app   # (gdb) tui enable ; start ; next ; bt
lldb ./build/app  # (lldb) b main ; run ; n ; bt
# tests and history
ctest --test-dir build -N              # ✓ list, run none
git log --oneline -20 -- path/to/area  # ✓
git shortlog -sn HEAD -- path/to/area  # ✓ who knows it
```

Lots of CMake, Python, or shell in `cloc` means the build is part of the problem. Zero docs means you will lean on generated docs (S10). Other entry points to hunt: static initializers (run before main), event handlers, thread start routines, plugin `register` calls, `subscribe` calls on a message bus.

DOCS Talk slides repo →https://github.com/philsquared/cpponsea2025-slides OpenTTD (the practice codebase) →https://www.openttd.org/ ctest(1) →https://cmake.org/cmake/help/latest/manual/ctest.1.html

4 SEARCH, NAVIGATE, HISTORY

USE WHEN you ask "Where is it?", "Who touches it?", or "When did it change?" Text search finds *spelling*. It cannot tell a read from a write.

STRATEGY

1. **Search the visible thing first**: the exact error text, log line, UI label, CLI flag. Users see strings, not identifiers.
2. **Classify hits, do not read them.** Split writers from readers with a regex.
3. **Over about 20 hits? Stop.** Go to the debugger (S7).
4. **Need callers?** Use clangd *incoming calls*, not more grep.
5. **Need the "why"?** Read the commit message, not just the diff.
6. **Regression?** `git bisect run` : about 7 test runs for 100 commits.

`RG` WITH NO PATH HANGS IN SCRIPTS. If stdin is not a terminal, ripgrep waits to read it. Always end with a path (`.` or `src/`). The writers regex also matches the declaration `int m_total = 0;` ; ignore that hit.

DYNAMIC DISPATCH HIDES FROM GREP. Virtual calls, callbacks, `std::function` , and message buses only show their truth at runtime. Break in the callee and read `bt` .

✓ RUN ON THIS MAC

```
grep -irn -I --include='*.cpp' "main()" .   # the talk's
form
rg -n --type cpp '\bmain\s*\(' .
rg -n -w m_total src/          # whole word
rg -n -C3 'applyDelta' src/    # context
rg -l 'Bus::post' src/         # files only
rg -n 'TODO|FIXME|HACK|XXX' src/
rg -n -F 'Connection refused' src/   # literal
# WRITERS of a member (textual only)
rg -n 'm_total\s*([-+*/%|&^]|<<|>>)?=[^=]' src/
rg --files . | rg -i cmake      # ALWAYS give a path (see
warning)
```

✓ RUN ON THIS MAC

```
# history: who, when, why
git log --oneline -20 -- path/
git log -S'applyDelta' --oneline   # string count changed
git log -G'm_total\s*\+=' -p       # diff matches regex
git log -L :applyDelta:src/tracker.cpp   # one function's
history
git blame -L 10,30 src/tracker.cpp
git blame -w -C -C src/tracker.cpp # follow moved code
git show SHA                       # read the message!

# regression: let git find the commit.
# Keep check.sh OUTSIDE the repo.
cat > /tmp/check.sh <<'EOF'
#!/bin/sh
cmake --build build -j >/dev/null 2>&1 || exit 125  # 125 =
skip
./build/app          # exit 0 good, 1-127 bad
EOF
chmod +x /tmp/check.sh
git bisect start HEAD KNOWN_GOOD_TAG
git bisect run /tmp/check.sh   # names first bad commit
git bisect reset               # ALWAYS finish with this
```

FROM DOCS · TRY ONCE FIRST

```
# semantic navigation: export real compile flags
cmake -S . -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
ln -sf build/compile_commands.json .
# editor: go to definition / find references /
# call hierarchy (incoming).  No LSP?
ctags -R --c++-kinds=+p --fields=+iaS src/  # Ctrl-] in vim
```

DOCS ripgrep guide →https://github.com/BurntSushi/ripgrep/blob/master/GUIDE.md git-bisect →https://git-scm.com/docs/git-bisect git-log →https://git-scm.com/docs/git-log clangd →https://clangd.llvm.org/

5 BUILD SYSTEM AND DEBUG SYMBOLS

USE WHEN the debugger shows `??` or `<optimized out>`, CMake is a maze, or "it works in release." The talk's loudest tip: build with debug information.

TWO SEPARATE DIALS

SETTING	MEANING
`-g -O0`	**Debug.** Easiest to debug.
`-g -O2`	**RelWithDebInfo.** Ship-like plus symbols. Use when the bug only shows in release.
`-O2`	**Release.** No symbols. `assert` compiled out (`-DNDEBUG`).
`-g -Og`	Optimized but debuggable. A good middle.

"WORKS IN RELEASE" PROOF (RUN ON THE SANDBOX).

Debug build: `Assertion failed`, exit 134. Release build: `total = -10`, exit **0**. The bug is still there; the `assert` was compiled out. A green Release build proves nothing if the check was an assert.

STRATEGY

1. Debug info is separate from optimization. Add `-g` to the build you ship, if that is the one that fails.
2. Make the compiler tell you its real command (`--verbose`), not what you think the CMake says.
3. 3rd-party libraries must also be built with symbols, or you see assembly when you step into them.
4. One build directory per configuration: `build/`, `build-rel/`, `build-asan/`.
5. Write the working build recipe in the journal the first time it works.

✓ RUN ON THIS MAC

```
# configure + build with symbols and a compile database
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug \
      -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
cmake --build build -j
# SEE the real compiler command (flags, -g, -O, -D)
cmake --build build --verbose
# what is this configured with?
cmake -LAH -N build | less
grep -E 'CMAKE_BUILD_TYPE|CXX_FLAGS' build/CMakeCache.txt
cmake --build build --target help     # what can I build?
cmake --build build --target app -j   # one target
# map of the maze: target dependency graph
cmake --graphviz=build/deps.dot build       # writes
deps.dot
```

FROM DOCS · TRY ONCE FIRST

```
dot -Tsvg build/deps.dot -o build/deps.svg   # needs
graphviz installed
cmake -S . -B build-rwdi -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake -S . -B build --trace-expand --trace-
source=CMakeLists.txt
cmake --build build --config Debug   # multi-config
generators
cmake -S . -B build -G Ninja         # faster builds
cmake -DCMAKE_CXX_COMPILER_LAUNCHER=ccache -S . -B build
```

DO I REALLY HAVE SYMBOLS?

MIXED · MARKED INLINE

```
dsymutil build/app && dwarfdump --debug-info build/app.dSYM
| head  # macOS ✓
# (dwarfdump on the bare macOS binary prints nothing: info
lives in .dSYM)
file build/app          # Linux: "with debug_info, not
stripped"
readelf --debug-dump=info build/app | head
# Linux
nm build/app | c++filt | grep Foo
# ✓
# in gdb:  info sources | list main | "No symbol table" =
none
# Separate debuginfo (stripped prod binary):
gdb -ex 'symbol-file /path/app.debug' -ex 'attach PID'
export DEBUGINFOD_URLS="https://debuginfod.elfutils.org/"
```

Five CMake commands cover 90% of reading: `add_executable/add_library`, `target_link_libraries`, `target_include_directories`, `target_compile_definitions/options`, `add_subdirectory`.

DOCS cmake(1) →https://cmake.org/cmake/help/latest/manual/cmake.1.html CMAKE_BUILD_TYPE →https://cmake.org/cmake/help/latest/variable/CMAKE_BUILD_TYPE.html Red Hat: all about debuginfo →https://developers.redhat.com/articles/2022/01/10/gdb-developers-gnu-debugger-tutorial-part-2-all-about-debuginfo Undo: symbols guide →https://undo.io/resources/guide-to-symbols-debug-info/ debuginfod →https://sourceware.org/elfutils/Debuginfod.html

6 DEBUGGER FUNDAMENTALS

USE WHEN you need to see what the program does: run, stop, step, inspect. A debugger is a code-understanding tool, not only a bug finder. Land on `main`, then follow what you care about.

ACTION	GDB (LINUX)	LLDB (MACOS)
Start	`gdb ./app` · `gdb --args ./app -x 5`	`lldb ./app` · `lldb -- ./app -x 5`
Attach to running process	`sudo gdb -p PID`	`sudo lldb -p PID`
Crash dump	`gdb ./app core`	`lldb ./app -c core`
Run to main and stop	`start`	`b main` then `run`
Pause a running program	`Ctrl-C`	`Ctrl-C`
Breakpoint	`b Foo::bar` · `b file.cpp:42`	`b Foo::bar` · `b -f file.cpp -l 42`
Conditional breakpoint	`b Bus::post if m.value < 0`	`b -n Bus::post -c 'm.value < 0'` ✓
Continue / next / step / finish	`c` / `n` / `s` / `finish` (Enter repeats last)	`c` / `n` / `s` / `finish`
Backtrace · pick frame	`bt`, `bt full` · `frame 2`, `up`, `down`	`bt` ✓ · `frame select 2`, `up`
Locals · print	`info locals` · `p *ptr`, `p/x n`, `p arr[0]@10`	`frame variable` ✓ · `p *ptr`
Type of a thing	`ptype obj` · `whatis x`	`p -T -- obj`
Kill, stay in debugger · quit	`kill` · `q`	`process kill` · `q`
Script it	`gdb -batch -x cmds.gdb ./app`	`lldb -b -s cmds.lldb ./app` ✓

STRATEGY

1. `start` to land at the entry point; step with `n` to see what happens before the real work. Use `s` into the "real main".
2. **Attach** to skip startup and dive in *after* something interesting happened. Needs `sudo` (or ptrace permission).
3. `finish` gets you out of library code you do not own.
4. Climb to the first frame in *your* code and print there.
5. `ptype` once you find an object: see the struct.
6. Use `set var` / `return` to *falsify* a hypothesis in seconds. These are experiments, not fixes.

SCOPE TRAP. LLDB conditions run in the frame that stopped. A name from `main` is "undeclared" inside another method. Use `this->m_total` or a raw address. (Found while testing.)

FROM DOCS · TRY ONCE FIRST

```
(gdb) tui enable                 # source view; the talk's favourite
(gdb) condition 3 m.value < 0    # add condition to bp #3
(gdb) ignore 3 99                # skip next 99 hits
(gdb) tbreak file.cpp:120        # one-shot
(gdb) rbreak ^Tracker::          # every method of a class
(gdb) dprintf file.cpp:42,"v=%d\n", v   # log, no stop, no rebuild
(gdb) commands 3
  silent
  bt 3
  continue
end
(gdb) info threads / thread apply all bt
(gdb) catch throw                # stop where exception is thrown
(gdb) info sharedlibrary         # code you don't own
(gdb) set var m_total = 0 / return 5 / call foo(3)
```

✓ RUN ON THIS MAC

```
# LLDB, run against the sandbox
breakpoint set -n Bus::post -c 'm.value < 0'
breakpoint list
breakpoint modify -c 'm.value < 0' 1
breakpoint set -r '^Tracker::'   # regex: whole class (7 locations)
frame variable m
```

DOCS GDB manual →https://sourceware.org/gdb/current/onlinedocs/gdb/ GDB to LLDB map →https://lldb.llvm.org/use/map.html LLDB tutorial →https://lldb.llvm.org/use/tutorial.html GDB cheat sheet PDF →https://darkdust.net/files/GDB%20Cheat%20Sheet.pdf How debuggers work (AOSA) →https://aosabook.org/en/v2/gdb.html

7 FOLLOW THE DATA

USE WHEN one variable shows up 100 times across methods, pointers, and a message queue. This is the sheet for your Raytheon problem. Do not read the code. Watch the memory.

THE IDEA

A **hardware watchpoint** makes the CPU stop the instant one address is written. It does not care which of the 100 textual occurrences did it, or whether it went through a pointer, a reference, `memcpy`, or a message handler. **The backtrace at that stop is the answer to "who touched it?"**

STRATEGY: SIX STEPS

1. **Get the live object** (an address, not a name). Break after construction; `p obj`, `p &obj.field`, `ptype obj`.
2. **Watch it with a condition** so you only stop on the bad write.
3. **Read the stop.** Frame 0 is the writer; the arguments show the wrong value; the stack shows the path.
4. **Hit a queue?** After it, `bt` shows who *delivered*, not who *sent*. Move the breakpoint to the **producer** (`post / send / enqueue / emit`) and filter on the bad payload.
5. **Which write was first?** Record and `reverse-continue` (S8).
6. **Many hits, no need to stop?** Attach `commands` to log each write plus `bt 3`, then grep the log.

> **RULE:** break at the producer, not the consumer. Filter on the bad payload. In the sandbox this one breakpoint lands in `Controller::rebalance`, the planted bug, without reading it.

> **HARDWARE LIMITS.** Typically 4 watchpoints, each up to 8 bytes (LLDB here: "supported hardware watchpoints: 4"). A watched local ends when its frame returns. A software fallback is very slow: watch one `int`, not a struct.

FROM DOCS · TRY ONCE FIRST

```
# GDB: -l watches the ADDRESS (survives scope change)
(gdb) break main.cpp:17        # object exists, bug not yet
(gdb) run
(gdb) p tracker  ;  ptype tracker  ;  p &tracker.m_total
(gdb) watch -l tracker.m_total
(gdb) watch -l tracker.m_total if tracker.m_total < 0
(gdb) rwatch -l tracker.m_total   # who READS it
(gdb) awatch -l tracker.m_total   # reads or writes
(gdb) continue
(gdb) break Bus::post if m.value < 0   # catch the sender
(gdb) break Tracker::applyDelta if this->m_id == 7
# if GDB can't evaluate the condition, use this->field
# or *(int*)0xADDR instead of a local's name
```

✓ RUN ON THIS MAC

```
# LLDB on practice-sandbox: all of this was run
(lldb) b -f main.cpp -l 17
(lldb) run
(lldb) watchpoint set variable tracker.m_total
(lldb) watchpoint modify -c 'this->m_total < 0' 1   # NOT
'tracker.m_total'
(lldb) continue
Watchpoint 1 hit:  old value: 0  new value: -10
* frame #0: Tracker::applyDelta(this=..., v=-10)
tracker.cpp:14
  frame #1: Tracker::onMessage(...)
tracker.cpp:9
  frame #9: Bus::pump                              bus.hpp:26
# dispatcher

(lldb) breakpoint set -n Bus::post -c 'm.value < 0'
(lldb) run
* frame #0: Bus::post(m=(type = Delta, value = -10))
  frame #1: Controller::rebalance(target=5)
controller.cpp:12  # SENDER
```

WHICH DATA TOOL?

YOU KNOW	YOU WANT	USE
Variable is wrong *now*	who wrote it	`watch -l` plus a condition
Wrong value arrives via a queue, bus, or callback	the original sender	conditional **break at post/send**
It already crashed or finished wrong	who wrote it *last*	`reverse-continue` (S8)
Who depends on it	consumers	`rwatch` / `awatch`
A pointer is dangling or null	who freed or nulled it	`watch -l ptr` + `bt`, or ASan (S12)
Only some of N instances are bad	the bad one	break with a condition on `this` or an id field
The write is in a hot loop	skip boring hits	`condition`, `ignore N`, `until`

Static helper. `rg -n 'm_total\s*([-+*/%|&^]|<<|>>)?=[^=]' src/` lists the textual writers. Use it as a checklist to cross out once the watchpoint shows the real writer. Writers that never fire in the failing run do not matter for this bug. **Practice:** `handbook/practice-sandbox/`, 15 minutes: find the root cause with only a watchpoint and one conditional breakpoint, before reading `controller.cpp`.

DOCS GDB watchpoints →`https://sourceware.org/gdb/current/onlinedocs/gdb/Set-Watchpoints.html` GDB conditions →`https://sourceware.org/gdb/current/onlinedocs/gdb/Conditions.html` LLDB tutorial →`https://lldb.llvm.org/use/tutorial.html` Greg Law: Time Travel Debugging (follow-the-data demo) →`https://www.youtube.com/watch?v=qyGdk6QMpMY`

8 TIME-TRAVEL DEBUGGING

USE WHEN you ask "how did I get here?" or "who wrote this value *last*?" Record the run, then step and continue *backwards*, with the whole program state unwound.

STRATEGY

1. **Bad value found?** `watch -l` it, then `reverse-continue`. It stops at the previous write.
2. **Crash far from the cause** (corruption, use-after-free)? Reverse from the crash to the corrupting write.
3. **Intermittent or race?** Record the failing run *once* and replay it deterministically forever.
4. **Unfamiliar flow?** Record one feature (a click, a request) and walk it forward and back.
5. **Third-party library?** The recording shows every API call, the arguments, and the results; compare with its docs.
6. **Record late**, near the bug, unless your recorder is fast.

TOOL	WHERE	NOTES
GDB `record full`	Linux	Free. Very slow; fine for tight loops.
rr	Linux x86	Free. Low overhead. Needs CPU perf counters (poor in VMs).
UDB / LiveRecorder	Linux	Commercial, free trial. About 1.5 to 10x slowdown. Same GDB commands.
WinDbg TTD	Windows	Visual Studio ecosystem.

APPLE SILICON: none of these run natively. Use a Linux box or VM. Ask what your team already has licensed.

FROM DOCS · TRY ONCE FIRST

```
# A) GDB built-in
(gdb) break main.cpp:17
(gdb) run
(gdb) record full               # start recording HERE
(gdb) continue                  # run until the failure
(gdb) watch -l tracker.m_total
(gdb) reverse-continue          # stops at the previous
WRITE
(gdb) bt                        # the writer, as it was then
(gdb) reverse-next / reverse-step / reverse-finish
(gdb) record stop

# B) rr
rr record ./build/app --case 42
rr replay                       # then the same gdb commands
rr record --chaos ./build/app   # randomize scheduling:
shakes out races

# C) UDB (the talk's tool)
nm build/app | grep MainLoop    # find the mangled symbol
live-record --record-on symbol:_Z12PostMainLoopv ./build/app
udb app-1234-DATE.undo          # replay; reverse-*
available
```

THREE RECIPES TO MEMORIZE

```
1 Who wrote this value last?
  watch -l OBJ.FIELD -> reverse-continue -> bt
2 How did I get here with this argument?
  break FUNC if ARG==BAD -> continue -> up -> reverse-
continue
3 Where did this pointer become null or get freed?
  watch -l PTR -> reverse-continue -> bt
```

Loop until captured: `while rr record ./app; do :; done` (stops when the failing run is recorded).

DOCS GDB reverse execution →https://sourceware.org/gdb/current/onlinedocs/gdb/Reverse-Execution.html rr →https://rr-project.org/ UDB free trial →https://undo.io/udb-free-trial/ GDB ReverseDebug wiki →https://sourceware.org/gdb/wiki/ReverseDebug

9 PROFILERS AND CALL GRAPHS

USE WHEN "what actually runs when I do X?" or "where is the time going?" A profiler also draws a map of an unfamiliar program: its hot paths, its main loop, its threads.

STRATEGY

1. **Explore:** start the app idle, start profiling, do *exactly one* action, stop. The report is that feature's code.
2. **Perform:** profile a release-like build (`RelWithDebInfo`), never `-O0` .
3. Sort by **self** time for hot loops, by **inclusive** for hot subsystems.
4. A hot function is *where to look*, not the bug. Take its name to the debugger and set a breakpoint on it.
5. Screenshot the call graph or flame graph into the journal.

INLINING HIDES FUNCTIONS. In a test the hot function showed up as `main` because it was inlined at `-O1` . Use `-fno-inline` or `-O0` when you want the graph to look like the source.

PERF SAYS "RESTRICTED"? Check `sysctl kernel.perf_event_paranoid` . Ask IT before changing it.

FROM DOCS · TRY ONCE FIRST

```
# perf (Linux): the talk's commands
sudo perf record -g ./build/app
sudo perf report
sudo perf record -p $(pgrep -n app) -g -- sleep 20   # attach
sudo perf report --no-children --stdio | head -60
sudo perf top -p $(pgrep -n app)     # live, like top
perf stat -d ./build/app             # counters

# flame graph (one-picture orientation)
sudo perf record -F 99 -g -p PID -- sleep 30
sudo perf script | ./FlameGraph/stackcollapse-perf.pl \
                 | ./FlameGraph/flamegraph.pl > flame.svg

# callgrind + KCachegrind (20-50x slower)
valgrind --tool=callgrind ./build/app
kcachegrind callgrind.out.PID
callgrind_annotate callgrind.out.PID | head -40
valgrind --tool=callgrind --instr-atstart=no ./build/app &
callgrind_control --instr=on   # do the action ...
callgrind_control --instr=off
```

✓ RUN ON THIS MAC

```
# macOS: sample works on your own processes, no sudo
sample $(pgrep -n app) 5 -file /tmp/sample.txt
less /tmp/sample.txt             # "Call graph:" per thread
```

FROM DOCS · TRY ONCE FIRST

```
# Instruments from the CLI. Needs FULL Xcode, not just
# Command Line Tools (it errored here for that reason).
xctrace record --template 'Time Profiler' --output app.trace
\
        --time-limit 10s --launch -- ./build/app
```

KCachegrind: left list shows inclusive/self cost and the **called** column; top treemap box size = time; Call Graph tab shows what really called what.

DOCS perf wiki →https://perfwiki.github.io/main/ Brendan Gregg: perf →https://www.brendangregg.com/perf.html FlameGraph →https://github.com/brendangregg/FlameGraph callgrind →https://valgrind.org/docs/manual/cl-manual.html KCachegrind usage →https://kcachegrind.sourceforge.net/html/Usage.html

10 STRUCTURE, DOCS, DEPENDENCIES

USE WHEN you need a map (classes, libraries, includes), you hit "undefined reference", or a library you do not own misbehaves. No docs? Generate them.

STRATEGY

1. **Generate docs** with Doxygen. Seeing the same code drawn differently builds a mental model.
2. Read them for **conventions**: return types, snake_case or camelCase, error codes or exceptions. This pays off at code review and tells you "my code or a library?"
3. **Name the pattern.** A message bus is Observer / publish-subscribe: who subscribes, who posts, where is the dispatch loop?
4. **Library you do not own:** (1) ask for a debug build; (2) record the run and compare API calls to the docs; (3) intercept at the boundary; (4) disassembly is a last resort.
5. **Offline docs** (Zeal, Dash) beat web search: no latency, fuzzy search.

MIXED · MARKED INLINE

```
doxygen -g                  # writes Doxyfile
doxygen Doxyfile            # writes html/
open html/index.html        # xdg-open on Linux
# Doxyfile: make it useful for undocumented code
RECURSIVE=YES  EXTRACT_ALL=YES  EXTRACT_PRIVATE=YES
HAVE_DOT=YES   CALL_GRAPH=YES   CALLER_GRAPH=YES
CLASS_GRAPH=YES  COLLABORATION_GRAPH=YES  GENERATE_LATEX=NO

cmake --graphviz=build/deps.dot build   # ✓ target graph
clang++ -std=c++17 -MM -Isrc src/foo.cpp   # ✓ headers
foo.cpp includes
clang++ -H -fsyntax-only -Isrc src/foo.cpp 2>&1 | head   # ✓
include tree
rg -n '#include\s+"bus.hpp"' src/          # who includes me
```

CODE YOU DO NOT OWN

QUESTION	LINUX	MACOS
Libraries needed	`ldd app` · `readelf -d app | grep NEEDED`	`otool -L app` ✓
Symbols defined	`nm -C --defined-only app`	`nm app | c++filt` ✓
Symbols needed	`nm -C -u app`	`nm -u app | c++filt`
Exports of a lib	`nm -D --defined-only -C libfoo.so`	`nm -gU libfoo.dylib`
Which lib got loaded?	`LD_DEBUG=libs ./app`	`DYLD_PRINT_LIBRARIE`
Windows	Dependency Walker · `dumpbin /dependents app.exe`	

FROM DOCS · TRY ONCE FIRST

```
# no source? watch the boundary
ltrace -f -e 'libfoo*' ./app          # library calls
strace -f -e trace=network ./app      # syscalls
(gdb) break libfoo_connect
(gdb) bt  ;  info args
# reformat a COPY to read it; lint patterns
clang-format -style=LLVM file.cpp > /tmp/readable.cpp
clang-tidy file.cpp -- -Isrc
```

DO NOT RUN `LDD` ON UNTRUSTED BINARIES. It can execute them. Use `readelf -d`.

DOCS Doxygen manual →https://www.doxygen.nl/manual/ Doxygen 1-minute demo →https://www.youtube.com/watch?v=tLPHQMosF9M Zeal →https://zealdocs.org/ Dash →https://kapeli.com/dash AOSA case studies →https://aosabook.org/en/ clang-tidy →https://clang.llvm.org/extra/clang-tidy/

11 SHRINK THE PROBLEM

USE WHEN the codebase is too big to hold in your head. Your task is not "understand the codebase." It is "understand *this path through* the codebase."

STRATEGY

1. **Look for a test of your component first.** A passing test is a tiny, deterministic program to run under the debugger.
2. **No test?** Write one that reproduces the bug. It is your repro *and* your regression test.
3. **Thin slice.** Find the smallest existing feature that behaves like yours. Trace it once end to end. Write the six-line path in the journal.
4. **Smallest change** that takes the test from red to green. Commit. Only then widen.
5. **Characterization tests** before touching scary legacy code: capture what it does *now*, then change it knowing what you broke.
6. **Good first tasks:** a labeled small issue, a compiler warning, change some text or a config value, add an item to a list. Each teaches one slice.
7. **Review your diff first:** every changed line should trace to the requirement.

MINIMIZE A REPRO

FROM DOCS · TRY ONCE FIRST

```
git bisect start BAD GOOD && git bisect run ./check.sh  # history
head -n 500 input.log > half.log && ./app half.log     # input: halve, back off
git diff --stat; git diff                              # self-review
```

✓ RUN ON THIS MAC

```
# find tests
find . -type d \( -name test -o -name tests -o -name unittests \) -not -path './build/*'
rg -l 'TEST(_F|_P)?\(|TEST_CASE\(|BOOST_AUTO_TEST_CASE' --glob '!build*' .
rg -n 'add_test|gtest_discover_tests|enable_testing' --glob 'CMakeLists.txt' .
# CTest (all run on the sandbox)
ctest --test-dir build -N          # list only
ctest --test-dir build -R tracker -V
ctest --test-dir build --output-on-failure
ctest --test-dir build --rerun-failed --output-on-failure
# find first tasks
rg -n -i 'TODO|FIXME|XXX|HACK|later|workaround|hard-?coded' --type cpp src/
```

FROM DOCS · TRY ONCE FIRST

```
rg -n -i 'tracker' tests/            # tests mentioning MY class
git log --since=3.months --oneline -- path/ | head -30
./tests --gtest_list_tests
./tests --gtest_filter='TrackerTest.*'
./tests --gtest_filter='TrackerTest.Reset*' --gtest_repeat=100  # flaky?
./tests --list-tests ; ./tests "[tracker]"        # Catch2
gdb --args ./tests --gtest_filter='TrackerTest.Reset*'
lldb -- ./tests --gtest_filter='TrackerTest.Reset*'
```

DOCS ctest(1) →https://cmake.org/cmake/help/latest/manual/ctest.1.html GoogleTest →https://google.github.io/googletest/ Catch2 →https://github.com/catchorg/Catch2 OpenTTD: how the talk found first tasks →https://www.openttd.org/development

12 CRASHES, MEMORY, HANGS, RACES

USE WHEN segfault, abort, garbage values, leak, freeze, or "only sometimes." Beyond the talk, but it is the next thing you will meet. Core rule: **CRASH SITE IS NOT BUG SITE.**

TRIAGE BY SYMPTOM

SYMPTOM	FIRST MOVE
`SIGSEGV`	debugger `bt`, then ASan
`terminate called after throwing`	`bt` ; `catch throw` for the origin
Garbage values, works on my machine	UBSan + ASan; valgrind `--track-origins=yes` ; `-Wall -Wextra`
Memory grows	LeakSanitizer / `leaks` / valgrind
Frozen, 0% CPU	attach, `thread apply all bt`
Frozen, 100% CPU	`perf top` / `sample`, then `bt`
Only sometimes; changes when you add a print	TSan; `rr --chaos` ; fewer threads
Fine in Debug, broken in Release	UB, uninitialized var, or `assert` doing real work: RelWithDebInfo + sanitizers

READING A CRASH

1. Frame 0 is where it *died*, usually innocent. `up` to the first frame in **your** code.
2. Print the pointer. Ask "who last set this or freed it?"
3. Then `watch -l ptr` (S7), `reverse-continue` (S8), or ASan's "freed by thread T0 here" stack.

FROM DOCS · TRY ONCE FIRST

```
ulimit -c unlimited       # allow core files in THIS shell
cat /proc/sys/kernel/core_pattern
coredumpctl list ; coredumpctl gdb
gdb ./build/app core      # bt full ; thread apply all bt
# macOS crash reports: ~/Library/Logs/DiagnosticReports/
(gdb) catch throw         # lldb: breakpoint set -E cxx
```

MIXED · MARKED INLINE

```
# sanitizers: separate build dir. Lines marked ✓ were run:
# "heap-buffer-overflow ... WRITE of size 4 ... main
leak.cpp:2"
cmake -S . -B build-asan -DCMAKE_BUILD_TYPE=Debug \
  -DCMAKE_CXX_FLAGS="-fsanitize=address,undefined -fno-omit-
frame-pointer -g"
cmake --build build-asan -j && ./build-asan/app
clang++ -g -O0 -fsanitize=address,undefined x.cpp -o x &&
./x   # ✓
# threads (cannot combine with ASan)
cmake -S . -B build-tsan -DCMAKE_CXX_FLAGS="-
fsanitize=thread -g"
# macOS leaks (reported "1 leak for 1536 total leaked
bytes")
MallocStackLogging=1 leaks --atExit -- ./build/app   # ✓
leaks $(pgrep -n app) ; heap $(pgrep -n app) | head
```

FROM DOCS · TRY ONCE FIRST

```
# valgrind (Linux, no rebuild)
valgrind --leak-check=full --track-origins=yes --error-
exitcode=1 ./app
valgrind --tool=helgrind ./app
# HANG: dump every thread's stack
sudo gdb -p $(pgrep -n app) -batch -ex 'thread apply all bt'
sudo eu-stack -p $(pgrep -n app)
sudo strace -f -p $(pgrep -n app)   # futex = lock wait
top -H -p $(pgrep -n app)           # which THREAD burns CPU
# macOS: sample PID 3 -file /tmp/s.txt   (tested)
lldb -p $(pgrep -n app) -o 'bt all' -o detach -o quit
```

✓ RUN ON THIS MAC

```
# race: make it deterministic, then loop until it fails
n=0; while ./build/app >/dev/null 2>&1; do n=$((n+1)); done
echo "failed after $n runs"
# rr record --chaos ./app   |   taskset -c 0 ./app
```

Deadlock signature: two threads each in `pthread_mutex_lock` / `futex_wait`, each holding what the other needs. Fix with lock ordering (`std::scoped_lock(a, b)`) or less sharing. A print "fixes" it? That is a race or UB, not a fix.

DOCS AddressSanitizer →https://clang.llvm.org/docs/AddressSanitizer.html ThreadSanitizer →https://clang.llvm.org/docs/ThreadSanitizer.html UBSan →https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html Valgrind →https://valgrind.org/docs/manual/manual.html strace(1) →https://man7.org/linux/man-pages/man1/strace.1.html core(5) →https://man7.org/linux/man-pages/man5/core.5.html

13 WORKSPACE AND JOURNAL

USE WHEN you want to see more at once, or you keep re-deriving yesterday's findings. Two force multipliers from the talk: a window manager, and a greppable journal.

STRATEGY

1. **More pixels, more context.** Ask for a big or portrait 4K monitor. Build and test are remote anyway.
2. **Split your terminal:** editor, debugger, search shell, and a journal window. tmux keeps it all alive if SSH drops.
3. **Use what works for you.** Do not adopt a tool because someone evangelized it.
4. **Journal as you go**: lowest-friction, greppable text. Carmack's `.plan` files are the model.
5. **Shadow a senior for 15 minutes.** Note the tools you saw.

DEFENSE-CONTRACTOR NOTE. Keep the journal where your program approves work notes. Export-controlled or program-specific details do not go in personal cloud notes or public repos. This handbook's generic method notes are fine; code excerpts and program details are not. Ask your lead.

MIXED · MARKED INLINE

```
# tmux (prefix Ctrl-b). Session/window creation ✓; keys from docs
tmux new -s debug ; tmux ls ; tmux attach -t debug
# %  split side/side    "  split top/bottom    o  next pane
# z  zoom pane    c  new window    n/p  next/prev    ,  rename
# d  detach (everything keeps running)    [  scroll mode (q)
# vim code-reading keys
# Ctrl-o / Ctrl-i  jump back/forward    *  search word under cursor
# gd local def    Ctrl-] tag    :vsplit f    Ctrl-w w    gf    :e +123 f
# :set grepprg=rg\ --vimgrep  then  :grep X  :copen  :cn
```

JOURNAL ENTRY TEMPLATE

```
## DATE TIME - one-line goal
Symptom:  exact text / value / exit code
Repro:    command, input, commit SHA, build dir
Hypotheses:
  1. falsifiable sentence -> test ran -> CONFIRMED /
     FALSIFIED because ...
Evidence: bt / log lines / call-graph screenshot path
Root cause (5 whys):
Fix: commit/PR     Regression test: name
Learned (tool/idea):          Open questions:
```

✓ RUN ON THIS MAC

```
# append-only daily journal (add to ~/.zshrc)
mkdir -p ~/swe-excellence/journal
jrnl() { printf '\n## %s  %s\n' "$(date '+%Y-%m-%d %H:%M')" "$*" \
  >> ~/swe-excellence/journal/$(date +%F).md
  ${EDITOR:-vim} + ~/swe-excellence/journal/$(date +%F).md; }
# find old notes:
rg -n 'm_total' ~/swe-excellence/journal/
```

CAPTURE ON DAY ONE IN A NEW AREA

- Working build recipe
- Entry points; what happens before and after `main`
- Key data structures (`ptype` output)
- A call-graph or flame-graph picture
- Questions asked, and who answered
- Which tools are approved at work

DOCS tmux wiki →https://github.com/tmux/tmux/wiki vim help →https://vimhelp.org/ The Carmack plan →https://garbagecollected.org/2017/10/24/the-carmack-plan/ MIT Missing Semester →https://missing.csail.mit.edu/

14 AI: TRUST, BUT VERIFY

USE WHEN an AI tool (or a confident colleague) gives you a root cause. From the talk: "It sounds authoritative. It's not. It's just good at being convincing."

FIRST: IS IT ALLOWED? Use only enterprise-licensed tools your organization has approved for this data. Program code, designs, logs, and error text may be export-controlled (ITAR/EAR) or CUI. Pasting them into a consumer chatbot can be a reportable incident. If unsure, ask security. Practise on this sandbox or open-source repos instead.

THE WORKFLOW: AI PROPOSES, DEBUGGER JUDGES

1. AI claims "the bug is X because Y."
2. Turn the claim into **one falsifiable experiment** (below).
3. Run it. Evidence agrees: keep it, journal it.
4. Evidence disagrees: give the AI the evidence ("the watchpoint never fired there; the write came from frame Z") and ask for a *new* hypothesis.

The talk's MySQL case: the AI confidently proposed wrong root causes and a "definitive patch." Time-travel debugging *constrained* it by forcing it to confirm every hypothesis with evidence.

GOOD AT / BAD AT

- **Good:** summarizing a file, searching fast, explaining unfamiliar code, driving tools
- **Bad:** knowing it is wrong, staying on track, challenging you, defending claims with evidence

TURN A CLAIM INTO AN EXPERIMENT

AI SAYS	RUN THIS
"`m_total` goes negative in `applyDelta`"	`watch -l ... if ... < 0`; does it stop there, with what `v`? (S7)
"Called once per frame"	`break Foo::tick` + `ignore 1 999` + `info breakpoints` hit count (S6, S9)
"Pointer is freed before use"	ASan: does it report use-after-free, with which stacks? (S12)
"This commit introduced it"	`git bisect run` (S4)
"These classes are not coupled"	`rg` + clangd incoming calls + Doxygen caller graph (S10)

PROMPTS THAT WORK (APPROVED TOOLS, SAFE CODE)

```
Orient: "Explain the control flow from main() to
FEATURE. Cite file:line for each claim. Mark
anything inferred rather than read as INFERRED."

Evidence: "Give ONE gdb/lldb command sequence that
would prove your hypothesis wrong. No fix yet."

Constrain: "Here is the bt and watchpoint output.
Which of your hypotheses does this falsify?"

Challenge: "What is the strongest alternative
explanation I have not excluded?"
```

Avoid: "fix this" with no repro; accepting the first explanation of a stack trace; letting it edit code before you can reproduce the failure. Tools named in the talk: Sourcegraph, ChatDBG, Claude Code, Undo's UDB + AI prototype.

DOCS Undo: TTD tames complex codebases and overconfident AI →https://undo.io/resources/time-travel-debugging-tames-complex-codebases-and-overconfident-ai/ ChatDBG →https://github.com/plasma-umass/ChatDBG Sourcegraph →https://sourcegraph.com/

15 LINKS AND NEXT STEPS

USE WHEN you want the primary source. Everything below is also in `handbook/chapters/15-resources-and-links.md`.

THE TALK

- Slides repo: github.com/philsquared/cpponsea2025-slides
- Session page: cpponsea.uk/2025/session/understanding-large-and-unfamiliar-codebases
- Greg Law, Time Travel Debugging (Meeting C++ 2023): youtube.com/watch?v=qyGdk6QMpMY
- Greg Law, Back to Basics: Debugging in C++: youtube.com/watch?v=qgszy9GquRs
- Mike Shah, Back to Basics: Debugging: youtube.com/watch?v=YzIBwqWC6EM
- Greg Law, GDB: A Lot More Than You Knew: youtube.com/watch?v=-n9Fkq1e6sg
- GDB cheat sheet: darkdust.net/files/GDB Cheat Sheet.pdf
- How a debugger works: aosabook.org/en/v2/gdb.html

Your notes link the video as `watch?v=qmte-ZrzBKUm`. That ID has one character too many, so it likely will not resolve. Search the title on the C++ on Sea channel.

OFFICIAL TOOL DOCS

GDB	sourceware.org/gdb/current/onlinedocs/gdb
LLDB	lldb.llvm.org
rr	rr-project.org
perf	perfwiki.github.io/main
Valgrind	valgrind.org/docs/manual
Sanitizers	clang.llvm.org/docs/AddressSanitizer.html
CMake	cmake.org/cmake/help/latest
ripgrep	github.com/BurntSushi/ripgrep GUIDE
git	git-scm.com/docs
cppreference	en.cppreference.com

BOOKS WORTH OWNING

Debugging: The 9 Indispensable Rules (Agans) · *Working Effectively with Legacy Code* (Feathers) · *Design Patterns* (Gang of Four) · *Why Programs Fail* (Zeller) · *The Linux Programming Interface* (Kerrisk)

PRACTICE, IN ORDER

1. **Sandbox, 15 min:** `handbook/practice-sandbox/`. Find the planted bug with one watchpoint and one conditional breakpoint.
2. **OpenTTD or SQLite, 1 hour:** do the S3 checklist, then trace one action with the debugger.
3. **Your task at work:** S3, then S11 (is there a test?), then S7 on the variable. Ask what tools are approved first.